1. Technical Context and Scenario

You are the founding ML engineer at a fintech startup deploying computer vision models for document fraud detection. The model classifies scanned documents as Genuine, Tampered, or Forged. The system is under active red-teaming and regulators require demonstrable adversarial robustness before go-live.

Your mandate covers four areas:

- Train a CNN classifier on a proxy dataset and establish a clean-accuracy baseline.
- Attack the model using multiple threat models and quantify robustness degradation.
- Implement and benchmark at least two defenses using the Adversarial Robustness Toolbox (ART).
- Build a RAG chatbot so non-technical stakeholders can interrogate model behavior, attack results, and experiment logs in natural language.

> **Proxy Dataset**
> Use **CIFAR-10** as the proxy dataset. Map the 10 original classes into 3 super-classes: Genuine (airplane, automobile, ship, truck), Tampered (bird, cat, deer, dog), and Forged (frog, horse). Justify your mapping choice in the README.

2. Task Breakdown

Task A | CNN Model Design and Training (PyTorch)

Design and train a convolutional neural network from scratch using PyTorch. This model serves as the target for all downstream adversarial experiments.

A1. Architecture Requirements

- Minimum 4 convolutional layers with batch normalisation and dropout.
- Justify filter sizes and depth choices in code comments.
- The model must achieve at least 75% clean accuracy on the held-out test split before you proceed to adversarial tasks.
- Use a custom PyTorch Dataset class and DataLoader pipeline with augmentation (random crop, horizontal flip, colour jitter).

A2. Training Setup

- Optimizer: Adam or AdamW with a cosine annealing learning rate schedule.
- Loss: Cross-entropy. Log train and validation loss and accuracy per epoch.
- Save the final checkpoint in MLflow as a registered model artifact.

A3. MLflow Experiment Tracking

- Create a dedicated MLflow experiment named AML-CNN-BASELINE.
- Log: all hyperparameters, per-epoch metrics, confusion matrix as an artifact, and the model checkpoint under the registered model name FraudCNN.
- Use MLflow autolog where applicable; override manually where finer control is needed.

```
# Minimal MLflow scaffolding expected
import mlflow, mlflow.pytorch

mlflow.set_experiment('AML-CNN-BASELINE')
with mlflow.start_run(run_name='baseline_cnn'):
    mlflow.log_params({'lr': 1e-3, 'epochs': 50, 'dropout': 0.3})
    # ... training loop ...
    mlflow.log_metrics({'val_acc': val_acc, 'val_loss': val_loss}, step=epoch)
    mlflow.pytorch.log_model(model, artifact_path='model')
```

Task B | Adversarial Attacks (ART)

Using IBM's Adversarial Robustness Toolbox, mount a systematic attack campaign against the trained CNN. All attacks must be wrapped in ART estimators for consistency.

B1. Required Attack Implementations

- **FGSM (Fast Gradient Sign Method):** Sweep epsilon in [0.01, 0.05, 0.1, 0.2]. Plot accuracy-vs-epsilon curve.
- **PGD (Projected Gradient Descent):** 20-step and 40-step variants. Compare attack strength.
- **C&W L2 Attack:** Run on a stratified sample of 200 images per class. Report mean L2 distortion and success rate.
- **Patch Attack (optional, bonus):** Implement a universal adversarial patch and demonstrate transfer to a second victim model.

B2. ART Estimator Wrapping

```
from art.estimators.classification import PyTorchClassifier
from art.attacks.evasion import FastGradientMethod, ProjectedGradientDescent,
CarliniL2Method

classifier = PyTorchClassifier(
    model=model,
    loss=criterion,
    optimizer=optimizer,
    input_shape=(3, 32, 32),
    nb_classes=3,
    clip_values=(0.0, 1.0)
)

# Log each attack run in its own MLflow child run
with mlflow.start_run(run_name='pgd_40step', nested=True):
    x_adv = pgd.generate(x=x_test)
    acc_adv = np.mean(np.argmax(classifier.predict(x_adv), axis=1) == y_test)
    mlflow.log_metric('adv_accuracy', acc_adv)
```

B3. Robustness Metrics to Report

- Clean accuracy vs adversarial accuracy per attack type.
- Average perturbation magnitude (L-inf and L2 norms).
- Attack success rate (ASR) and fooling rate.
- Per-class vulnerability analysis (which of the 3 classes is most susceptible?).

Task C | Decision Boundary Visualization

Visualize how the CNN partitions the feature space and how adversarial perturbations push samples across class boundaries.

C1. 2D Projection Techniques (implement at least 2)

- **UMAP or t-SNE:** Project the penultimate layer activations of clean and adversarial test samples. Plot clean vs adversarial embeddings side by side, coloured by true class and predicted class.
- **PCA with Decision Regions:** Reduce to 2 principal components and use mlxtend or a custom matplotlib loop to render decision regions. Overlay clean and adversarial sample positions.
- **Linear Interpolation Boundary Probe:** For 20 clean-adversarial pairs, plot the model confidence along the linear path between them. Identify the boundary crossing point.

C2. Boundary Distance Analysis

- Compute the minimum L2 distance from each test sample to the nearest decision boundary using ART's BoundaryAttack or a DeepFool variant.
- Report mean boundary distance per class and correlate with per-class attack success rates from Task B.
- Discuss in the README whether the model is uniformly robust or has class-specific weak spots.

Task D | Adversarial Defense

Implement and benchmark defenses using ART. Compare effectiveness vs the attacks from Task B on the same evaluation suite.

D1. Required Defenses (implement both)

- **Adversarial Training:** Fine-tune the baseline CNN using ART's AdversarialTrainer with PGD-generated training examples. Run for at least 10 additional epochs and log in MLflow experiment AML-DEFENSE.
- **Feature Squeezing:** Apply median smoothing and bit-depth reduction as pre-processing defenses. Wrap with ART's FeatureSqueezing preprocessor. Tune and justify squeezing parameters.

D2. Optional Defense (bonus)

- Certified defense: Implement Randomized Smoothing (Cohen et al., 2019) using ART's SmoothClassifier. Report certified accuracy at radius r = 0.25.

D3. Defense Comparison Table

Produce a defense comparison report (logged as an MLflow artifact) with the following columns: Model Variant, Clean Accuracy, FGSM Accuracy (eps=0.05), PGD-40 Accuracy, C&W Success Rate, and Training Overhead (hours).

Task E | RAG Chatbot for Model Explainability

Build a retrieval-augmented generation chatbot that allows non-technical stakeholders to ask natural language questions about your model, experiments, and adversarial findings. The chatbot must answer from your actual experiment outputs, not from general ML knowledge.

E1. Knowledge Base Construction

- Export all MLflow experiment runs (params, metrics, tags, artifact summaries) to structured text files using the MLflow tracking API.
- Include the README, defense comparison table, boundary distance analysis, and per-class attack results as source documents.
- Chunk documents at the section level (not sentence level). Record chunk source metadata for citation in answers.

E2. RAG Pipeline

```
# Suggested minimal RAG stack
# Embeddings: sentence-transformers (all-MiniLM-L6-v2 or similar)
# Vector store: ChromaDB or FAISS (local, no cloud dependency)
# LLM: any open-source model via Hugging Face (Mistral-7B-Instruct recommended)
#       OR: Anthropic Claude API (claude-sonnet-4-6) if API key available

# The pipeline must:
# 1. Embed the query
# 2. Retrieve top-k relevant chunks (k=5 minimum)
# 3. Construct a prompt with retrieved context + query
# 4. Return answer WITH source citations (document name + section)
```

E3. Chatbot Interface

- Implement a CLI chatbot (required) and optionally a Gradio or Streamlit UI (bonus).
- The chatbot must handle at least the following question types:
  - Performance questions: 'What is the clean accuracy of the baseline model?'
  - Attack questions: 'Which attack caused the most accuracy degradation?'
  - Defense questions: 'How much did adversarial training improve robustness against PGD?'
  - Boundary questions: 'Which class is closest to the decision boundary on average?'
  - Comparison questions: 'Compare feature squeezing vs adversarial training on C&W attacks.'

E4. Hallucination Guard

- If the chatbot cannot find sufficient evidence in the retrieved chunks, it must respond with: 'Insufficient evidence in experiment logs. Please consult raw MLflow runs.' It must NOT hallucinate metrics.
- Log 5 sample Q&A pairs (with retrieved chunks and final answers) in a chatbot_eval.json artifact in MLflow.

3. Deliverables

Deliverable	Format	Required
GitHub Repository	Public repo with README, requirements.txt, and clear run instructions	**Yes**
Trained Model Checkpoint	Saved in MLflow artifacts as registered model FraudCNN	**Yes**
Attack Evaluation Report	Markdown or PDF with accuracy tables and perturbation plots	**Yes**
Decision Boundary Visualizations	PNG/SVG plots (UMAP, PCA regions, boundary probe)	**Yes**
Defense Comparison Table	CSV or Markdown table, logged as MLflow artifact	**Yes**
RAG Chatbot (CLI)	Runnable chatbot_cli.py with a demo transcript	**Yes**
MLflow Experiment Exports	mlflow_export/ folder with all run JSONs/CSVs	**Yes**
1-Page Summary PDF	Architecture decisions, key findings, known limitations	**Yes**
chatbot_eval.json	5 sample Q&A pairs with retrieved chunks	**Yes**
Gradio/Streamlit UI	Optional chatbot web interface	Bonus
Universal Adversarial Patch	Patch visualization and transfer attack demo	Bonus
Certified Robustness (RS)	Randomized Smoothing implementation and accuracy at r=0.25	Bonus

4. Evaluation Rubric

Assessment Area	Key Criteria	Weight
CNN Design & Training	Architecture depth, augmentation, clean accuracy >=75%, proper DataLoader usage	**20%**
Adversarial Attacks (ART)	Correct ART estimator wrapping, all 3 required attacks, metric completeness	**25%**
Decision Boundary Analysis	Correctness of projections, boundary distance methodology, interpretation quality	**15%**
Adversarial Defense	Both defenses implemented, fair comparison methodology, defense efficacy	**20%**
RAG Chatbot	Answer accuracy from experiment logs, citation accuracy, hallucination guard, CLI demo	**15%**
MLflow Integration	Experiment naming, metric/param/artifact logging, registered model, nested runs	**5%**

5. Technical Constraints and Rules

5.1 Required Libraries

- **torch >= 2.1.0** and **torchvision >= 0.16.0** for model and dataset.
- **adversarial-robustness-toolbox >= 1.17.0** for all attacks and defenses.
- **mlflow >= 2.10.0** for experiment tracking.
- **sentence-transformers** and **chromadb** or **faiss-cpu** for RAG embeddings and retrieval.
- Any open-source LLM for generation (Hugging Face) or Anthropic Claude API.

5.2 Prohibited Approaches

- Do not use pre-trained ImageNet weights for the base CNN. Train from scratch.
- Do not use AutoAttack as a substitute for implementing individual attacks. You must wrap each attack manually in ART.
- Do not hard-code metrics into chatbot answers. All answers must originate from retrieved document chunks.
- Do not use LangChain or LlamaIndex as black-box wrappers for the RAG pipeline. Implement retrieval and prompt construction explicitly so the logic is visible.

5.3 Reproducibility

- All random seeds must be fixed (torch, numpy, random) and documented in a REPRODUCIBILITY.md file.
- A single shell script run_all.sh must execute the full pipeline from training through chatbot demo in sequence.
- Target hardware: single GPU (NVIDIA, >= 8 GB VRAM) or Google Colab T4. Document estimated wall-clock time.

6. Submission Instructions

Share the GitHub repository link (with collaborator access granted if private) and the 1-page summary PDF to the contact listed below before the 72-hour deadline. Late submissions will not be evaluated.

7. Reference Repository Structure

Your submission repository is expected to follow this layout. Deviations are allowed if clearly documented in the README.

```
adversarial-ml-assessment/
├── README.md                    # Setup, dataset mapping justification, key findings
├── REPRODUCIBILITY.md           # Seeds, environment, estimated runtimes
├── requirements.txt
├── run_all.sh                   # End-to-end pipeline script
|
├── data/
|   └── cifar10_loader.py        # Custom Dataset + DataLoader
|
├── models/
|   ├── fraud_cnn.py             # CNN architecture definition
|   └── checkpoints/             # Saved .pth files (gitignored, but listed in README)
|
├── attacks/
|   ├── fgsm_sweep.py
|   ├── pgd_attack.py
|   └── cw_attack.py
```

```
|
├── defenses/
|   ├── adversarial_training.py
|   └── feature_squeezing.py
|
├── visualization/
|   ├── boundary_probe.py
|   ├── umap_plot.py
|   └── pca_regions.py
|
├── rag/
|   ├── build_knowledge_base.py  # MLflow export + chunking + embedding
|   ├── retriever.py             # FAISS/Chroma vector store queries
|   ├── chatbot_cli.py           # CLI entrypoint
|   ├── chatbot_eval.json        # 5 Q&A pairs with chunks
|   └── gradio_app.py            # (optional) Gradio UI
|
├── mlflow_export/               # Exported run JSONs and metric CSVs
└── reports/
    ├── attack_evaluation.md
    ├── defense_comparison.csv
    └── summary.pdf              # 1-page submission summary
```

8. Viva Questions (Prepare to Answer)

These questions will be asked during the 30-minute code walkthrough. You will not be given them in advance (other than in this document). Prepare concise, precise answers backed by your experiment results.

On Attacks

- Why does PGD produce stronger adversarial examples than FGSM at the same epsilon, and what does this imply about the local geometry of the loss landscape near your CNN's decision boundary?
- Your per-class vulnerability analysis shows one class has a significantly lower adversarial accuracy. What structural or data-distribution reason might explain this?

On Defenses

- Adversarial training improves robustness but often reduces clean accuracy. Where did you observe this trade-off, and how would you tune it for a production fraud detection system where false negatives are costly?
- Feature squeezing is a pre-processing defense. Explain one adaptive attack that could circumvent it, and how ART could be used to evaluate your defense against such an adaptive adversary.

On Decision Boundaries

- What does the mean boundary distance per class tell you about the relative robustness of each class, and does it correlate with your attack success rates from Task B?

- Explain the boundary probe (linear interpolation) result for one specific clean-adversarial pair: at what fraction of the path does the predicted class change, and what does this suggest about the margin of the model?

On the RAG Chatbot

- Describe one specific failure mode you observed in the chatbot (e.g., a question it answered incorrectly or refused to answer), and explain whether it was a retrieval failure or a generation failure.
- How would you extend the chatbot to support real-time querying against a live MLflow tracking server rather than a static export?